pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Tree Instability and the Bootstrap

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

One tree, many possible trees

Week 1: Bagging

Average many trees fitted to bootstrap samples

Week 2: Boosting

Trees one after another, each fixing the last

Week 3: Tuning, rare events

Honest tuning, features, imbalance, SHAP

Today's question

If the training rows had been slightly different, how different would the tree be?

One set of rows, many plausible others: draw rows at random, with replacement, as many as there are.

A tree is unstable. An average of many unstable trees is not. That is week 1.

Where Course 4 left the tree

0.7190

logistic regression

0.6161

default tree (day 9)

0.7627

pruned tree (day 10)

Validation ROC AUC, Course 4's 18,000 training rows fit, its 6,000 validation rows report

The default tree

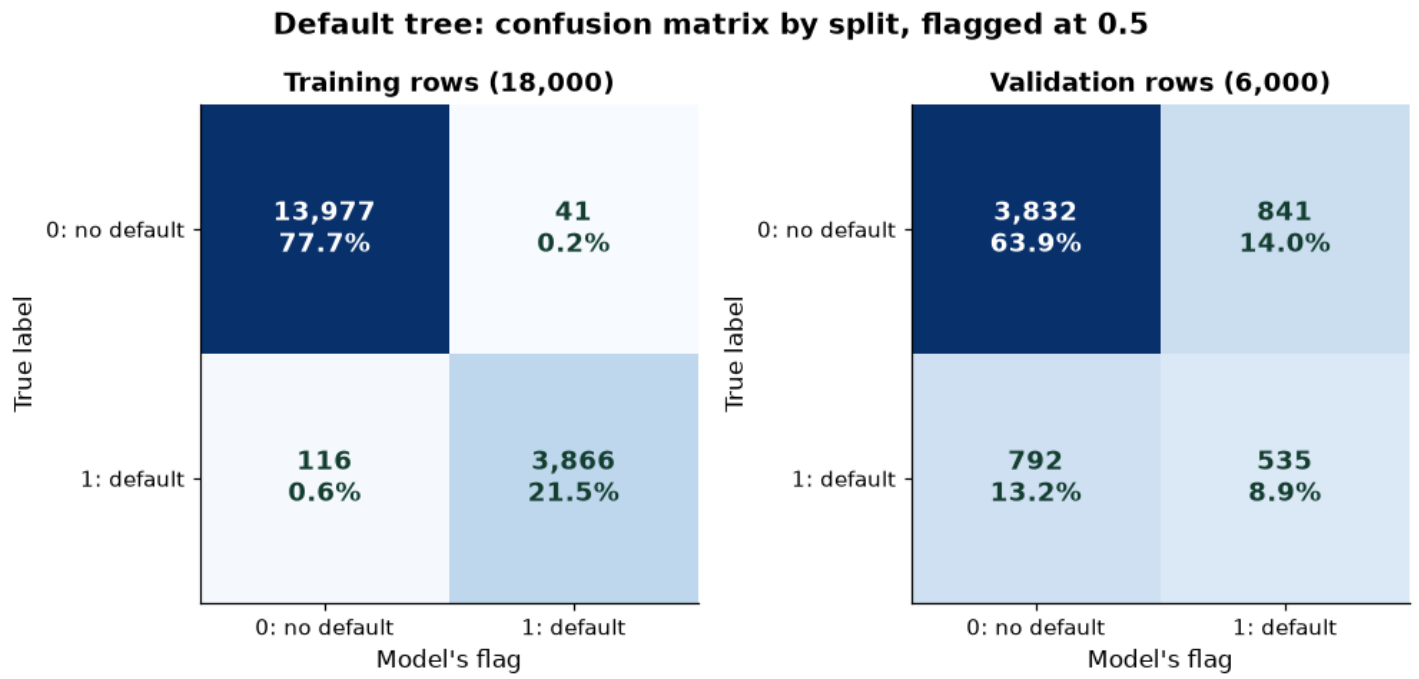
2,750 leaves, 38 questions deep. Training ROC AUC 0.9997: its rows, memorised.

The pruned tree

Class weights, ccp_alpha 0.001 by five folds: 12 leaves, 6 deep. Its output is the model's score.

These three numbers are the bar every Course 5 ensemble on the card file is judged against.

The standard view: one heatmap per split



Training, then validation, on every card day. The test rows never get a panel.

How to read it

True label down the side, the model's flag at 0.5 across the top. Each cell: a count and its share of that split's rows.

The default tree

Training panel: almost all on the diagonal (ROC AUC 0.9997). Validation panel: far from it (0.6161).

One rule for the whole course: the test rows stay closed



fit; choose by cross-validation

report only

closed

Why closed

Course 4 opened them once, on day 10. Scored again after their numbers were seen, they would become rows the course chooses against.

How it is enforced

course4_card_rows rebuilds Course 4's split and never returns a closed row. A test recomputes the closed labels and asserts none comes back.

Validation rows were used in Course 4 to choose, never to fit. Course 5 chooses nothing on them.

How a bootstrap sample works

The problem	One set of rows gives one fitted tree; a different set would give a different one
Draw	Pick a row at random, note it, put it back, and repeat until the sample is as big as the original
Repeats and gaps	Some rows come up twice or more, and some never come up
You control	The sample size, normally the original's, and the seed
What comes out	A list of row positions; the rows not listed are the sample's out-of-bag rows

The bootstrap: draw with replacement

18,000 draws from 18,000 rows, each row put back after it is drawn. 11,383 distinct rows made it in (63.24 percent); 6,617 never did.

Times drawn	Rows
0	6,617
1	6,662
2	3,236
3	1,150
4	268

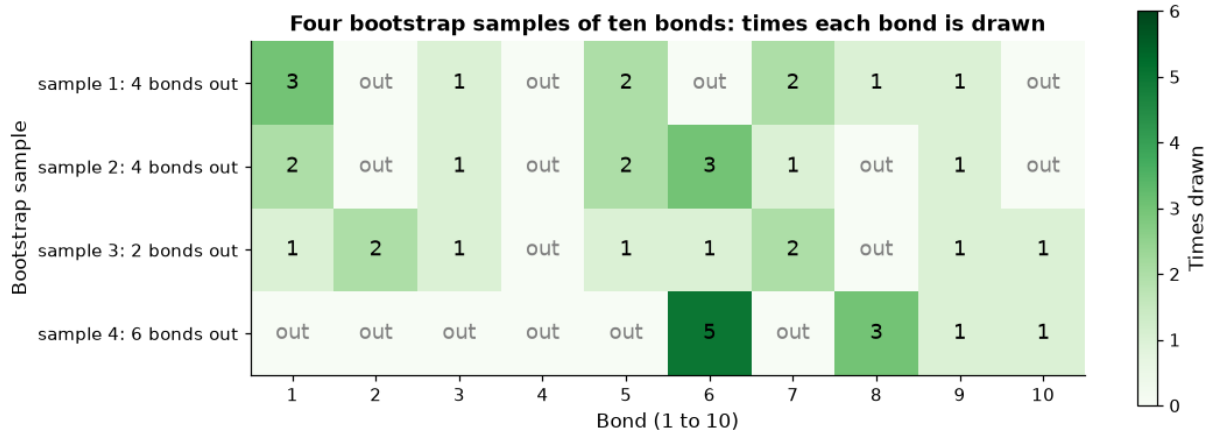
Why about 36.8 percent are left out

```
(1 - 1/n) ** n   ->   e ** -1   as n grows
```

At n = 18,000: 0.367869. e to the power -1: 0.367879. Each draw misses a row with chance 1 - 1/n, and the n draws are independent.

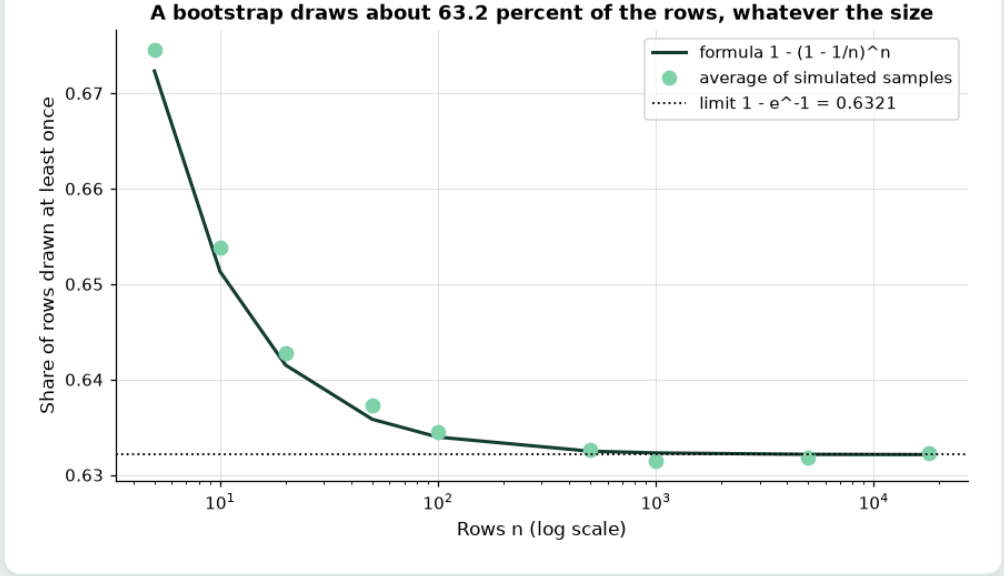
A bootstrap sample is the training rows with a random weight on each row.

A bootstrap you can see, at any size



Ten bonds, four samples

Bonds out of each sample: [4, 4, 2, 6]. Expected 3.49 = 10 x (1 - 1/10)^10.



The formula gives 0.6723 at 5 rows and 0.6321 at 18,000. Simulated samples sit on it.

About 37 percent out of bag holds for any bootstrap of a realistic size.

Ten samples, ten full trees

The first question, in all ten trees

```
PAY_0 <= 1.5
```

The same as Course 4's default tree. No reshuffle of the rows moves it.

1,832 to 1,921

leaves

35 to 44

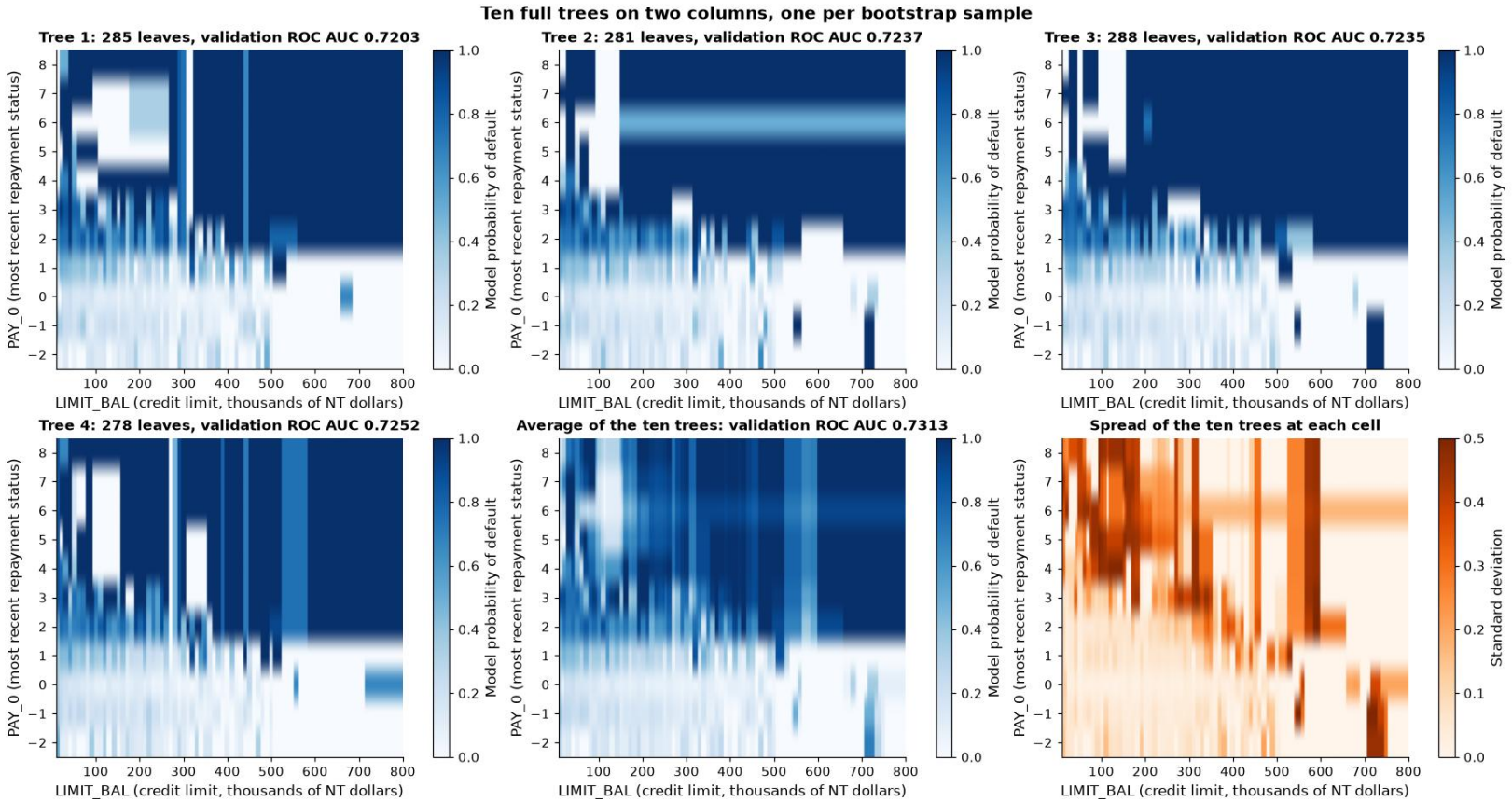
questions deep

0.6015 to 0.6219

validation ROC AUC

Deep in each tree, the splits follow the rows its sample drew twice or left out.

Where the ten trees disagree



What stays

The rows for a status of 2 or more are far darker than the rows for 1 or less: the first question.

What moves

Cut-offs along the credit limit. 27.6% of 2,200 cells spread above 0.2. The average ranks at 0.7313; single trees 0.7176 to 0.7252.

One account, ten outputs

97.7%

outputs exactly 0 or 1

0.2858

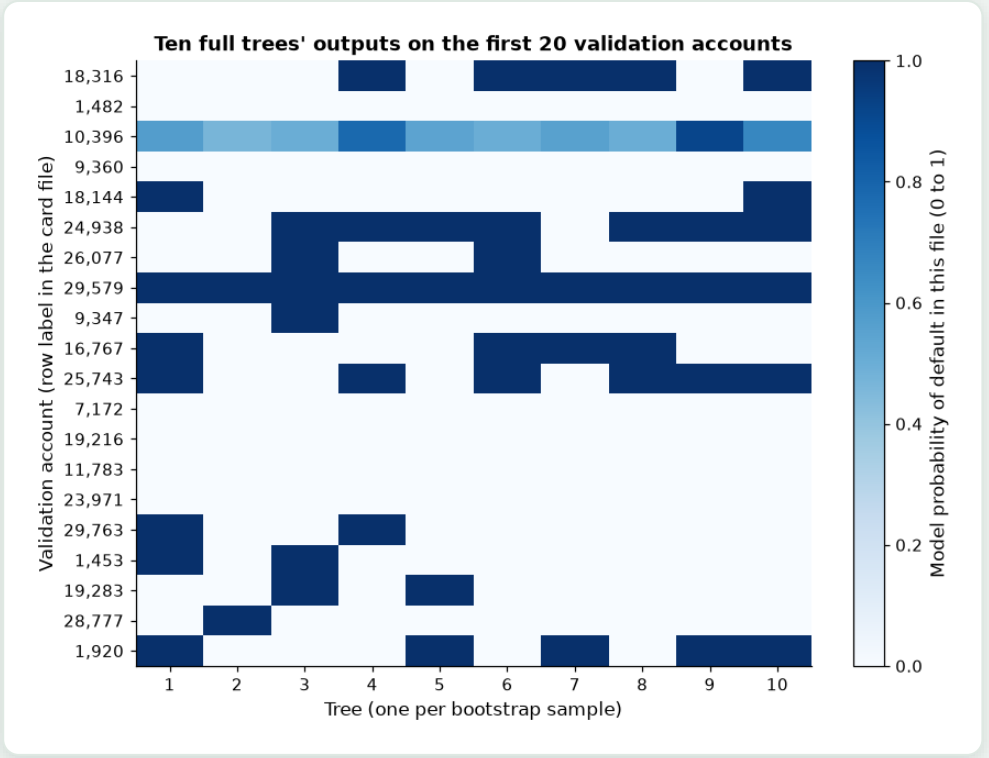
average spread

73.2%

flags that change

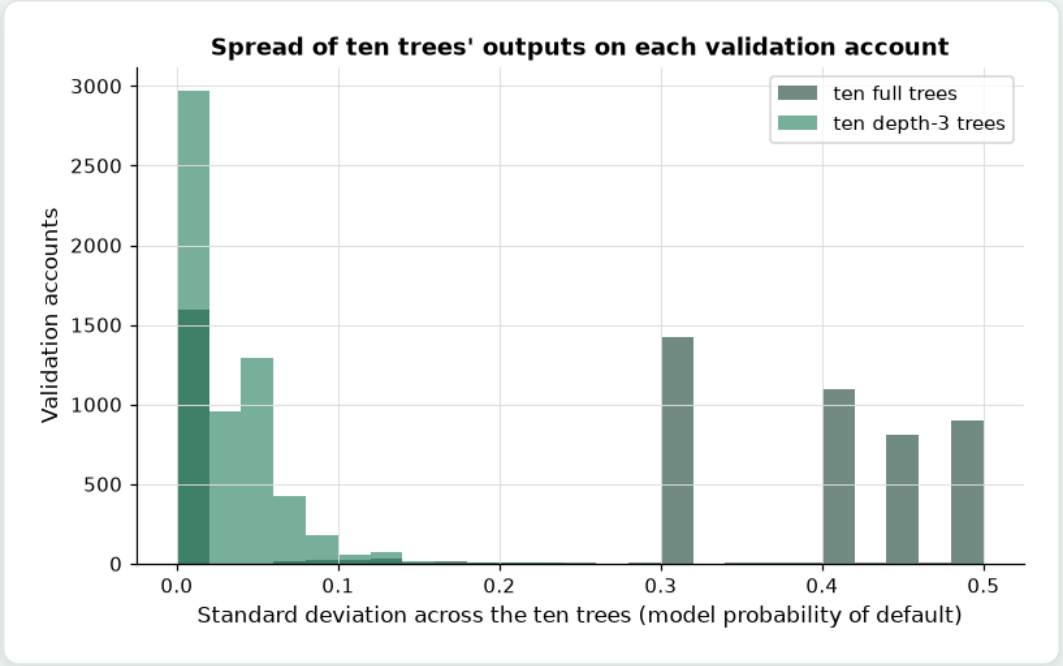
On 4,395 of 6,000 validation accounts, at least one tree flags the account at 0.5 and at least one does not.

Whether one full tree flags an account depends on which rows it happened to see.



Depth-3 trees: steadier, and weaker

Ten trees	Full	Depth 3
Validation ROC AUC	0.6015 to 0.6219	0.7109 to 0.7330
Average spread	0.2858	0.0306
Flag changes	73.2%	4.3%

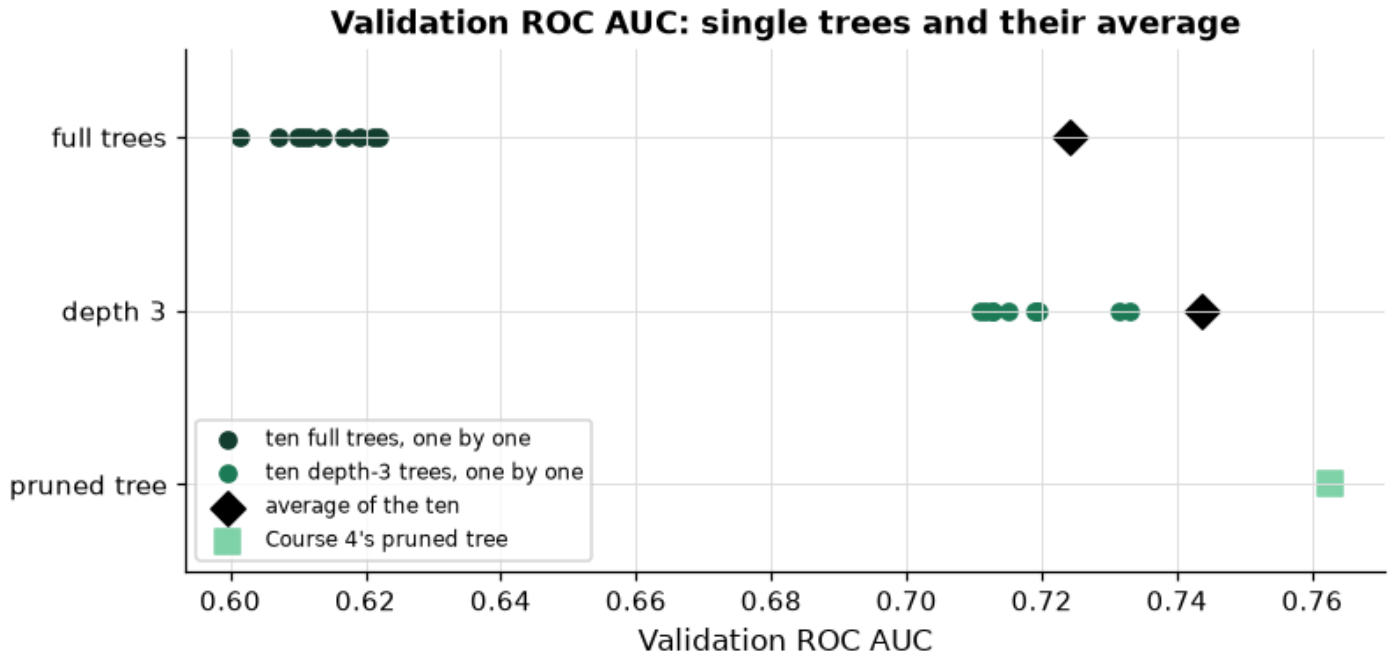


A depth limit buys steadiness by giving up detail: still below the pruned tree's 0.7627.

How averaging trees works

The problem	A full tree also fits its sample's accidents, so another sample gives another tree
Fit and score	One full tree per bootstrap sample; score every account with every tree
Average	The mean of the trees' outputs for each account: close to the share of trees that flag it
You control	How many samples to draw and how deep each tree grows
What comes out	One model probability of default in this file per account, ranked more steadily than any one tree

Averaging the ten full trees



0.7243

average of ten full trees

0.6219

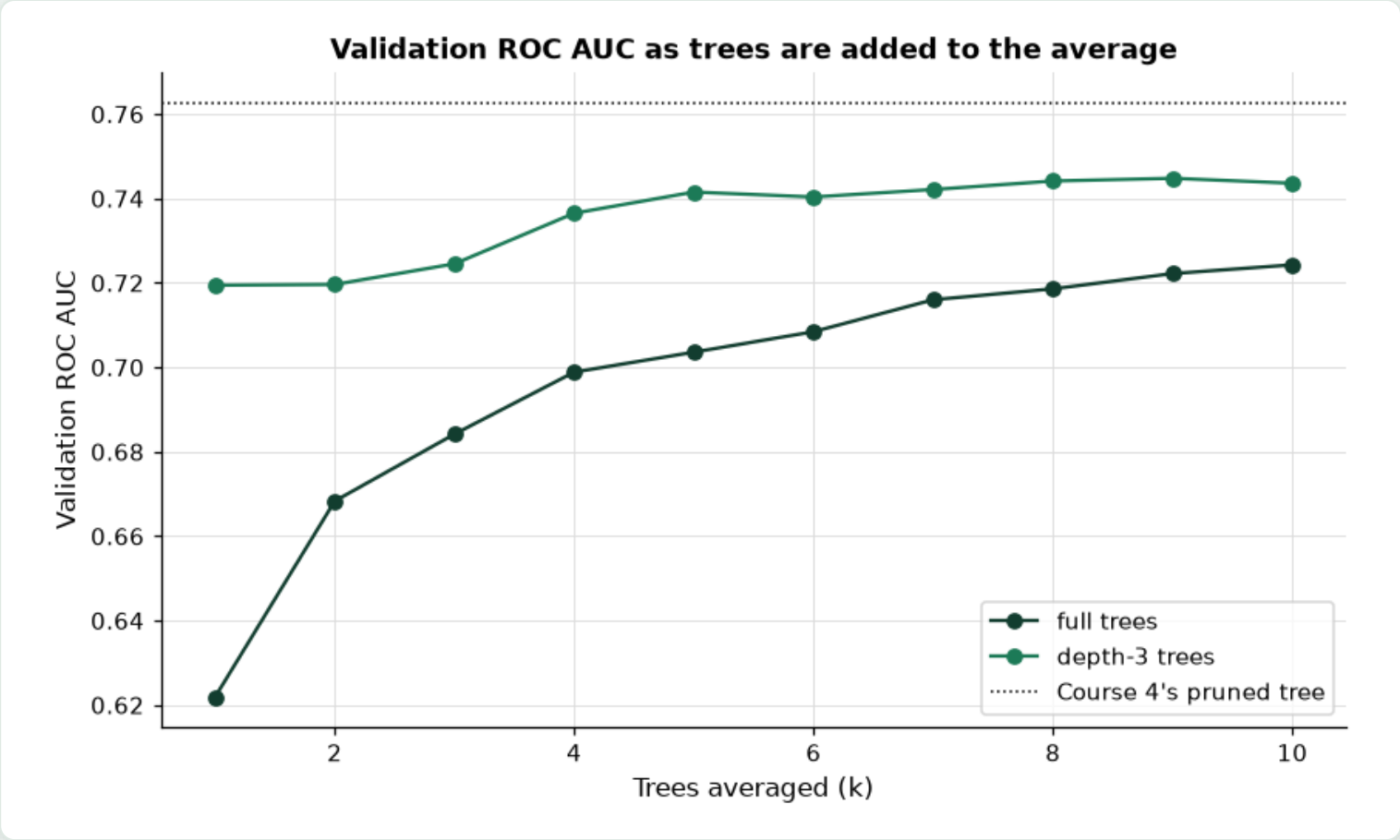
best single full tree

Depth 3, averaged

0.7436, against their best single tree's 0.7330: they already agree, so little cancels.

Averaging pays where the members disagree.

Adding trees one at a time



Trees	Full	Depth 3
1	0.6219	0.7195
10	0.7243	0.7436

The full-tree line climbs with every tree added; the depth-3 line moves far less. Neither reaches the pruned tree yet.

How many trees are enough? Day 2 continues the line to 100.

Where the name comes from

Leo Breiman, August 1996

"Bagging predictors"

Machine Learning, volume 24, issue 2, pages 123 to 140

The name this week uses: fit the same model to many bootstrap samples, and average.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1007/BF00058655). Bibliographic facts only; the paper itself was not read for this course.

How the out-of-bag rows work

The problem	A tree needs a score from rows it never saw, without spending the validation rows
Left out	A bootstrap sample leaves out about a third of the training rows
Score	Take every training row not on the sample's list and score the tree on those alone
You control	Nothing new: the seed that fixes the sample fixes the left-out rows
What comes out	The out-of-bag rows, and a score for one tree that costs no validation rows

The out-of-bag rows

The rows a sample never drew. The tree fitted on it never saw them, so they can score it without touching the validation rows.

6,617

tree 1 never saw

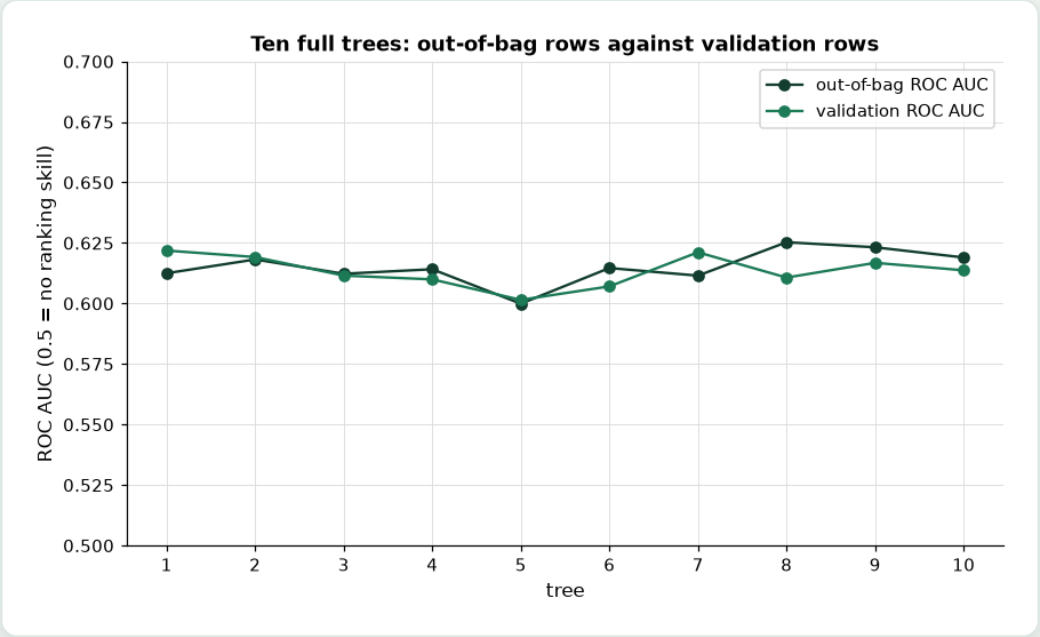
0.6125

on them

0.6219

on validation

```
ensemblekit.out_of_bag(n_rows, samples[0])
```



All ten trees: the two lines run together. Day 2 scores a whole bagged model this way. 7 tests pass today.

Excel side by side

Excel side by side (tested in Excel)

`=(1-1/n)^n`	0.367869 at n = 18,000
`=FREQUENCY(A2:A18001,SEQUENCE(18000,1,0))`	one count per position, in C2
`=COUNTIF($C$2:$C$18001,k)`	rows drawn k times: 6,617, 6,662, 3,236
`=ROWS(A2:A18001)-COUNTA(UNIQUE(A2:A18001))`	6,617 never drawn
`=STDEV.P(B2:K2), then =AVERAGE(L2:L6001)`	average spread 0.2858
`=AVERAGE(B2:K2)`	the ten-tree average
`=COUNTIFS(N2:N6001,">0",N2:N6001,"<10")/ROWS(N2:N6001)`	flag changes 0.7325

No seeded bootstrap and no tree in Excel. RAND, RANDBETWEEN, RANDARRAY take no seed and recalculate: the positions come from Python.

Save the day: the experiment log

1. Init

The work folder, local settings, the guard

2. .gitignore

__pycache__/, .pytest_cache/, *.xlsx

3. Code

mlkit.py with ensemblekit and its tests

4. Results

results.csv, 4 decimals

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
course4_default_tree,validation,0.6161,0.2917,0.5000,0.7278,0.3888,0.4032,0.3959
course4_pruned_tree,validation,0.7627,0.4954,0.5000,0.7683,0.4805,0.5840,0.5272
average_of_ten_full_trees,validation,0.7243,0.4453,0.5000,0.7907,0.5349,0.4099,0.4642
```

Threshold 0.5 today; from day 5, out-of-fold. Accuracy beside the all-zero baseline, 0.7788, never to choose.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the bootstrap is drawn with rng.permutation: every row once, no row out of bag.

Code that runs is not the same as code that is right.

What you can do now

- Rebuild Course 4's rows; keep its test rows closed
- Draw a bootstrap sample; about 36.8 percent left out
- A full tree's first question is stable; its leaves are not
- A depth limit buys steadiness and costs detail
- Averaging pays where the members disagree

Week 1, Day 2 Bagging with scikit-learn, and the out-of-bag score